

FAISS Documentation: Getting Started

Source: <https://github.com/facebookresearch/faiss/wiki/Getting-started>

For the following, we assume Faiss is installed. Code examples are given in Python.

Getting Some Data

Faiss handles collections of vectors of a fixed dimensionality d , typically a few 10s to 100s. These collections can be stored in matrices. We assume row-major storage, i.e. the j 'th component of vector number i is stored in row i , column j of the matrix. Faiss uses only 32-bit floating point matrices.

We need two matrices:

- x_b for the database, that contains all the vectors that must be indexed, and that we are going to search in. Its size is nb -by- d .
- x_q for the query vectors, for which we need to find the nearest neighbors. Its size is nq -by- d . If we have a single query vector, $nq=1$.

```
import numpy as np
d = 64                                # dimension
nb = 100000                           # database size
nq = 10000                            # nb of queries
np.random.seed(1234)                 # make reproducible
xb = np.random.random((nb, d)).astype('float32')
xb[:, 0] += np.arange(nb) / 1000.
xq = np.random.random((nq, d)).astype('float32')
xq[:, 0] += np.arange(nq) / 1000.
```

In Python, the matrices are always represented as numpy arrays. The data type dtype must be float32.

Building an Index and Adding the Vectors to It

Faiss is built around the Index object. It encapsulates the set of database vectors, and optionally preprocesses them to make searching efficient. There are many types of indexes; the simplest performs brute-force L2 distance search: IndexFlatL2.

All indexes need to know when they are built which is the dimensionality of the vectors they operate on, d in our case. Most indexes also require a training phase, to analyze the distribution of the vectors — for IndexFlatL2, we can skip this operation.

When the index is built and trained, two operations can be performed on the index: add and search.

To add elements to the index, call add on x_b . We can also display the two state variables of the index:

`is_trained`, a boolean that indicates whether training is required, and `ntotal`, the number of indexed vectors.

Some indexes can also store integer IDs corresponding to each of the vectors (but not IndexFlatL2). If no IDs are provided, add just uses the vector ordinal as the id — the first vector gets 0, the second 1, etc.

```
import faiss                          # make faiss available
index = faiss.IndexFlatL2(d)          # build the index
print(index.is_trained)
index.add(xb)                         # add vectors to the index
print(index.ntotal)
```

This should display `true` (the index is trained) and `100000` (vectors are stored in the index).

Searching

The basic search operation that can be performed on an index is the k-nearest-neighbor search: for each query vector, find its k nearest neighbors in the database.

The result of this operation can be stored in an integer matrix of size `nq-by-k`, where row `i` contains the IDs of the neighbors of query vector `i`, sorted by increasing distance. In addition, the search operation returns a `nq-by-k` floating-point matrix with the corresponding squared distances.

As a sanity check, we can first search a few database vectors, to make sure the nearest neighbor is indeed the vector itself.

```
k = 4                                # we want to see 4 nearest neighbors
D, I = index.search(xb[:5], k) # sanity check
print(I)
print(D)
D, I = index.search(xq, k)         # actual search
print(I[:5])                      # neighbors of the 5 first queries
print(I[-5:])                     # neighbors of the 5 last queries
```

Sample output of the sanity check:

```
[[ 0 393 363 78]
 [ 1 555 277 364]
 [ 2 304 101 13]
 [ 3 173 18 182]
 [ 4 288 370 531]]

[[ 0.          7.17517328  7.2076292  7.25116253]
 [ 0.          6.32356453  6.6845808  6.79994535]
 [ 0.          5.79640865  6.39173603  7.28151226]
 [ 0.          7.27790546  7.52798653  7.66284657]
 [ 0.          6.76380348  7.29512024  7.36881447]]
```

i.e. the nearest neighbor of each query is indeed the index of the vector itself, and the corresponding distance is 0. Within a row, distances are increasing.

Because of the value added to the first component of the vectors, the dataset is smeared along the first axis in `d`-dimensional space, so the neighbors of the first few vectors are around the beginning of the dataset, and the ones of the vectors around `~10000` are also around index `10000` in the dataset.

Source: FAISS Wiki, [facebookresearch/faiss](https://github.com/facebookresearch/faiss), © Meta Platforms, Inc.